Εθνικό Μετσόβιο Πολυτεχνείο

Εργαστήριο Μικροϋπολογιστών

4η Εργαστηριακή Άσκηση

Χριστόφορος Χαραλάμπους 03121614
Φίλιππος Γιαννακόπουλος 03121629

Ζήτημα 4.1

Το Ζήτημα 4.1 βρίσκεται στο αρχείο `erg4.1.asm`.

Συμπεριλαμβάνει τον κώδικα διαίρεσης `div16u` από την βιβλιοθήκη AVR200.

Η εισερχόμενη τάση υπολογίζεται με τον τύπο:

$$V_{\text{IN}} = \frac{\text{ADC}}{1024} V_{\text{REF}}$$

όπου $V_{\text{REF}} = 5V$.

Η διαίρεση με το 1024 γίνεται μέσω απλά της εξαγωγής των τελευταίων 10bit ως bit δεκαδικών ψηφίων. Τα υπόλοιπα είναι ο ακέραιος αριθμός.

Για την μετατροπή των δεκαδικών ψηφίων σε αναπαράσταση 0-99, πολλαπλασιάζουμε με 100 και διαιρούμε δια 1024. Ακολούθως, με ακέραια διαίρεση με το 10 παίρνουμε το πρώτο και το δεύτερο ψηφίο που πρέπει να παρουσιάσουμε.

```
.include "m328PBdef.inc"

.def drem16uL=r14
.def drem16uH=r15
.def dres16uL=r16
.def dres16uH=r17
.def dd16uL=r16
.def dd16uH=r17
.def dv16uL=r18
.def dv16uH=r19
.def dcnt16u=r20

.def BIN = r16          ; Input: 8-bit binary number
.def HUNDREDS = r17     ; Output: BCD hundreds digit
.def TENS = r18         ; Output: BCD tens digit
.def UNITS = r19        ; Output: BCD units digit

.equ FOSC_MHZ=16
.equ DEL_mS=1000
.equ F1=FOSC_MHZ*DEL_mS

.org 0x0
    rjmp reset

.org 0x2A
    rjmp ADC_DONE

reset:
    ; Initialize Stack Pointer
    ldi r24, LOW(RAMEND)
    out SPL, r24
    ldi r24, HIGH(RAMEND)
    out SPH, r24

    ; PORTD as output
    ser r24
    out DDRD, r24

    ; Setup LCD
    rcall lcd_init

    ; Setup ADC
    ldi r24, (1<<REFS0) | (1<<MUX0)
```

```
    sts ADMUX, r24

    ldi r24, (1<<ADEN) | (1<<ADIE) | (1<<ADPS2) | (1<<ADPS1) | (1<<ADPS0)
    sts ADCSRA, r24

    ; Setup wait time
    ldi r24, low(F1)
    ldi r25, high(F1)

    sei
main:
    lds r20, ADCSRA
    ori r20, (1<<ADSC)
    sts ADCSRA, r20
    rcall wait_x_msec
    rjmp main

ADC_DONE:
    push r20
    push r21
    push r22
    push r23
    push r24
    push r25

    lds r28, ADCL
    lds r29, ADCH
    mov r30, r28
    mov r31, r29

    ; Rotate left twice to multiply by 4
    clc
    rol r28
    rol r29
    clc
    rol r28
    rol r29

    ; Add one
    add r28, r30
    adc r29, r31

    ; Copy integer part and shift right twice
    mov r30, r29
    lsr r30
    lsr r30

    ; Isolate decimal part
    andi r29, 0b00000011

    ; Multiply by 100
    ldi r20, 100
    mul r28, r20

    mov r21, r0
    mov r22, r1
    mul r29, r20

    ldi r23, 0

    add r22, r0
```

```
    adc r23, r1

    ; Divide by 1024
    ldi r20, 10
repeat:
    clc
    ror r23
    ror r22
    ror r21
    dec r20
    brne repeat

    ldi dv16uL, LOW(10)
    ldi dv16uH, HIGH(10)
    mov dd16uL, r21
    mov dd16uH, r22

    rcall div16u

    mov r21, dres16uL
    mov r22, drem16uL

    ; BCD
    mov BIN, r30
    rcall binary_to_bcd
    ori TENS, 0b00110000
    ori UNITS, 0b00110000

    ; Display to LCD
    rcall lcd_clear_display

    mov r24, UNITS
    rcall lcd_data

    ldi r24, '.'
    rcall lcd_data

    ; BCD for Decimals
    mov BIN, r21
    rcall binary_to_bcd
    ori UNITS, 0b00110000
    mov r24, UNITS
    rcall lcd_data

    mov BIN, r22
    rcall binary_to_bcd
    ori UNITS, 0b00110000
    mov r24, UNITS
    rcall lcd_data

    ldi r24, 'V'
    rcall lcd_data

    pop r25
    pop r24
    pop r23
    pop r22
    pop r21
    pop r20
    reti
```

```
lcd_init:
    ldi r24, low(200*16)
    ldi r25, high(200*16)
    rcall wait_x_msec

    ; Switch to 8bit mode
    ldi r24, 0x30
    out PORTD, r24
    sbi PORTD, 3
    nop
    nop
    cbi PORTD, 3
    ; Wait 250usec = 0.25ms -> 0.25msec * 16 = 4
    ldi r24, HIGH(4)
    ldi r25, LOW(4)
    rcall wait_x_msec

    ; Switch to 8bit mode
    ldi r24, 0x30
    out PORTD, r24
    sbi PORTD, 3
    nop
    nop
    cbi PORTD, 3
    ; Wait 250usec = 0.25ms -> 0.25msec * 16 = 4
    ldi r24, HIGH(4)
    ldi r25, LOW(4)
    rcall wait_x_msec

    ; Switch to 8bit mode
    ldi r24, 0x30
    out PORTD, r24
    sbi PORTD, 3
    nop
    nop
    cbi PORTD, 3
    ; Wait 250usec = 0.25ms -> 0.25msec * 16 = 4
    ldi r24, HIGH(4)
    ldi r25, LOW(4)
    rcall wait_x_msec

    ; Switch to 4bit mode
    ldi r24, 0x20
    out PORTD, r24
    sbi PORTD, 3
    nop
    nop
    cbi PORTD, 3
    ; Wait 250usec = 0.25ms -> 0.25msec * 16 = 4
    ldi r24, HIGH(4)
    ldi r25, LOW(4)
    rcall wait_x_msec

    ; 5x8 dots, 2 lines
    ldi r24, 0x28
    rcall lcd_command

    ldi r24, 0x0c
    rcall lcd_command

    rcall lcd_clear_display
```

```
    ; Increase address, no display shift
    ldi r24, 0x06
    rcall lcd_command

    ret


lcd_clear_display:
    ldi r24, 0x01
    rcall lcd_command

    ldi r24, low(5*16)
    ldi r25, high(5*16)
    rcall wait_x_msec

    ret

lcd_data:
    sbi PORTD, 2      ; LCD_RS=1(PD2=1), Data
    rcall write_2_nibbles   ; send data
    ; Wait 250usec = 0.25ms -> 0.25msec * 16 = 4
    ldi r24, HIGH(4)
    ldi r25, LOW(4)
    rcall wait_x_msec
    ret

lcd_command:
    cbi PORTD, 2      ; LCD_RS=1(PD2=1), Data
    rcall write_2_nibbles   ; send data
    ; Wait 250usec = 0.25ms -> 0.25msec * 16 = 4
    ldi r24, HIGH(4)
    ldi r25, LOW(4)
    rcall wait_x_msec
    ret

write_2_nibbles:
    push r24

    in r25, PIND
    andi r25, 0x0F
    andi r24, 0xF0
    add r24, r25
    out PORTD, r24

    sbi PORTD, 3
    nop
    nop
    cbi PORTD, 3

    pop r24
    swap r24

    andi r24, 0xF0
    add r24, r25
    out PORTD, r24

    sbi PORTD, 3
    nop
    nop
    cbi PORTD, 3
```

```
    ret

wait_x_msec:
    push r23
    push r24
    push r25
repeat_x:
    rcall wait_one_msec
    sbiw r24,1
    brne repeat_x

    pop r25
    pop r24
    pop r23
    ret

wait_one_msec:
    ldi r23, 247
repeat_one:
    dec r23
    nop
    brne repeat_one

    nop
    ret

div16u: clr drem16uL  ;clear remainder Low byte
  sub drem16uH,drem16uH;clear remainder High byte and carry
  ldi dcnt16u,17  ;init loop counter
d16u_1: rol dd16uL    ;shift left dividend
  rol dd16uH
  dec dcnt16u   ;decrement counter
  brne  d16u_2    ;if done
  ret     ;    return
d16u_2: rol drem16uL  ;shift dividend into remainder
  rol drem16uH
  sub drem16uL,dv16uL ;remainder = remainder - divisor
  sbc drem16uH,dv16uH ;
  brcc  d16u_3    ;if result negative
  add drem16uL,dv16uL ;    restore remainder
  adc drem16uH,dv16uH
  clc     ;    clear carry to be shifted into result
  rjmp  d16u_1    ;else
d16u_3: sec     ;    set carry to be shifted into result
  rjmp  d16u_1

binary_to_bcd:
    ; Clear result registers
    clr HUNDREDS        ; Clear hundreds place
    clr TENS            ; Clear tens place
    clr UNITS           ; Clear units place

    ; Step 1: Extract Hundreds Digit
    ldi r20, 100        ; Load divisor (100)
loop_hundreds:
    cp BIN, r20         ; Compare BIN to 100
    brlo done_hundreds  ; If BIN < 100, skip to next step
    sub BIN, r20        ; BIN -= 100
    inc HUNDREDS        ; Increment hundreds digit
    rjmp loop_hundreds  ; Repeat until BIN < 100
```

```
done_hundreds:
    ; Step 2: Extract Tens Digit
    ldi r20, 10         ; Load divisor (10)
loop_tens:
    cp BIN, r20         ; Compare BIN to 10
    brlo done_tens      ; If BIN < 10, skip to next step
    sub BIN, r20        ; BIN -= 10
    inc TENS            ; Increment tens digit
    rjmp loop_tens      ; Repeat until BIN < 10
done_tens:
    ; Step 3: The remaining value in BIN is the Units Digit
    mov UNITS, BIN      ; Store remaining BIN value in UNITS

    ret                 ; Return with HUNDREDS, TENS, UNITS containing BCD digits
```

Ζήτημα 4.2

Το Ζήτημα 4.2 βρίσκεται στο αρχείο `erg4.2.c`.

Η άσκηση ακολουθεί όμοια λογική με την 4.1.

Γίνεται χρήση μεταβλητής τύπου `long` η οποία είναι 32bit για να έχουμε αρκετά bit για τον πολλαπλασιασμό που γίνεται με το 100.

```
#define F_CPU 16000000UL
#include <avr/io.h>
#include <util/delay.h>
#include <avr/interrupt.h>

unsigned char hundreds, tens, units;

void write2(unsigned char input) {
    unsigned char prev = PIND;

    PORTD = (input & 0xF0) | (prev & 0x0F);

    PORTD |= (1<<3);
    _delay_us(1);
    PORTD &= 0b11110111;

    PORTD = ((input & 0x0F) << 4) | (prev & 0x0F);

    PORTD |= (1<<3);
    _delay_us(1);
    PORTD &= 0b11110111;
}

void lcd_data(unsigned char input) {
    PORTD |= (1<<2);
    write2(input);
    _delay_us(250);
}

void lcd_command(unsigned char input) {
    PORTD &= 0b11111011;
    write2(input);
    _delay_us(250);
}

void lcd_clear() {
    lcd_command(0x01);
    _delay_ms(5);
}

void lcd_init() {
    _delay_ms(200);

    // Switch to 8bit mode
    PORTD = 0x30;
    PORTD |= (1<<3);
    _delay_us(1);
    PORTD &= 0b11110111;
    _delay_us(250);

    // Switch to 8bit mode
    PORTD = 0x30;
    PORTD |= (1<<3);
```

```
    _delay_us(1);
    PORTD &= 0b11110111;
    _delay_us(250);

    // Switch to 8bit mode
    PORTD = 0x30;
    PORTD |= (1<<3);
    _delay_us(1);
    PORTD &= 0b11110111;
    _delay_us(250);

    // Switch to 8bit mode
    PORTD = 0x20;
    PORTD |= (1<<3);
    _delay_us(1);
    PORTD &= 0b11110111;
    _delay_us(250);

    lcd_command(0x28);

    lcd_command(0x0c);

    lcd_clear();

    lcd_command(0x06);
}

void display_number(unsigned char input) {
    lcd_data(0b00110000 | (input & 0x0F));
}

void binary_to_bcd(unsigned char input) {
    hundreds = 0;
    tens = 0;

    while (input >= 100) {
        hundreds++;
        input -= 100;
    }

    while (input >= 10) {
        tens++;
        input -= 10;
    }

    units = input;
}

int main(void) {
    // PORTD as output
    DDRD = 0xFF;

    // Setup ADC
    ADMUX = (1<<REFS0) | (1<<MUX0);
    ADCSRA = (1<<ADEN) | (1<<ADPS2) | (1<<ADPS1) | (1<<ADPS0);

    lcd_init();

    while (1) {
        // Request ADC data
        ADCSRA |= (1<<ADSC);
```

```
        while ((ADCSRA & (1<<ADSC)) != 0);

        // Processing
        unsigned int input = ADC;
        unsigned int copy = input;

        copy = ADC<<2;
        copy += input;

        unsigned char integer = (copy>>10) & 0xFF;
        unsigned long decimal = copy & 0b0000001111111111;
        decimal *= 100;
        decimal = decimal>>10;

        // Display
        lcd_clear();

        binary_to_bcd(integer);
        display_number(units);
        lcd_data('.');

        binary_to_bcd(decimal);
        display_number(tens);
        display_number(units);

        lcd_data('V');

        // Repeat after 1s
        _delay_ms(1000);
    }
}
```

Ζήτημα 4.3

Το Ζήτημα 4.3 βρίσκεται στο αρχείο `erg4.3.c`.

Γίνεται χρήση Vref=3V στην `ISR(TIMER3_COMPA_VECT)` για την μετατροπή του αποτελέσματος του ADC, για να ταιριάζει με την τάση εξόδου που θα έδινε ο ULPSM-CO-968-001.

Σε αυτή την άσκηση, η επεξεργασία της εισόδου γίνεται με χρήση `double` για ευκολία στις πράξεις.

Για την παρουσίαση του επιπέδου του CO μέχρι τα 70ppm, ξεκινώντας από τα 0 και σβηστά όλα τα LED, κάθε 10ppm ανάβει ένα περισσότερο LED.

Ο υπολογισμός του επιπέδου CO γίνεται με:

$$C_X = \frac{1}{M} \cdot (\text{Vgas} - \text{Vgas}_0)$$

όπου $\text{Vgas}_0 = 0.1V$ και το M έχει υπολογιστεί ως εξής:

$$\begin{aligned} M &= \text{Sensitivity Code} \cdot \text{TIA Gain} \cdot 10^-9 \cdot 10^3 \\ &= 129 \cdot 100 \cdot 10^-9 \cdot 10^3 \\ &= 0.0129 \\ \frac{1}{M} &= 77.5194 \end{aligned}$$

```c
#define F_CPU 16000000UL
#include <avr/io.h>
#include <util/delay.h>
#include <avr/interrupt.h>

unsigned int last_measure = 0;

void write2(unsigned char input) {
    unsigned char prev = PIND;

    PORTD = (input & 0xF0) | (prev & 0x0F);

    PORTD |= (1<<3);
    PORTD &= 0b11110111;

    PORTD = ((input & 0x0F) << 4) | (prev & 0x0F);

    PORTD |= (1<<3);
    PORTD &= 0b11110111;
}

void lcd_data(unsigned char input) {
    PORTD |= (1<<2);
    write2(input);
    _delay_us(250);
}

void lcd_command(unsigned char input) {
    PORTD &= 0b11111011;
    write2(input);
    _delay_us(250);
}

void lcd_clear() {
    lcd_command(0x01);
    _delay_ms(5);
}
```

```
void lcd_init() {
    _delay_ms(200);

    // Switch to 8bit mode
    PORTD = 0x30;
    PORTD |= (1<<3);
    PORTD &= 0b11110111;
    _delay_us(250);

    // Switch to 8bit mode
    PORTD = 0x30;
    PORTD |= (1<<3);
    PORTD &= 0b11110111;
    _delay_us(250);

    // Switch to 8bit mode
    PORTD = 0x30;
    PORTD |= (1<<3);
    PORTD &= 0b11110111;
    _delay_us(250);

    // Switch to 8bit mode
    PORTD = 0x20;
    PORTD |= (1<<3);
    PORTD &= 0b11110111;
    _delay_us(250);

    lcd_command(0x28);

    lcd_command(0x0c);

    lcd_clear();

    lcd_command(0x06);
}

ISR(TIMER3_COMPA_vect) {
    ADCSRA |= (1<<ADSC);
    while ((ADCSRA & (1<<ADSC)) != 0);

    double input = ADC * 3;
    input = input / 1024;
    input = input - 0.1;
    input = input * 77.519;
    last_measure = input;
}

void display_gas_detected() {
    lcd_data('G');
    lcd_data('A');
    lcd_data('S');
    lcd_data(' ');
    lcd_data('D');
    lcd_data('E');
    lcd_data('T');
    lcd_data('E');
    lcd_data('C');
    lcd_data('T');
    lcd_data('E');
    lcd_data('D');
}
```

```
void display_clear() {
    lcd_data('C');
    lcd_data('L');
    lcd_data('E');
    lcd_data('A');
    lcd_data('R');
}

int main(void) {
    DDRD = 0xFF;
    DDRB = 0xFF;

    // Setup ADC
    ADMUX = (1<<REFS0) | (1<<MUX1);
    ADCSRA = (1<<ADEN) | (1<<ADPS2) | (1<<ADPS1) | (1<<ADPS0);

    // Timer 3 for interrupts to get DC data (CLK/64)
    TCCR3A = 0;
    TCCR3B = (0<<WGM33) | (1<<WGM32) | (0<<CS32) | (1<<CS31) | (1<<CS30);
    // 16MHz / (64 * 10Hz) - 1
    OCR3A = 24999;
    TIMSK3 = (1<<OCIE3A);

    lcd_init();

    sei();

    while (1) {
        if (last_measure <= 10) {
            PORTB = 0x00;
        } else if (last_measure <= 20) {
            PORTB = 0b00000001;
        } else if (last_measure <= 30) {
            PORTB = 0b00000011;
        } else if (last_measure <= 40) {
            PORTB = 0b00000111;
        } else if (last_measure <= 50) {
            PORTB = 0b00001111;
        } else if (last_measure <= 60) {
            PORTB = 0b00011111;
        } else if (last_measure <= 70) {
            PORTB = 0b00111111;
        } else {
            lcd_clear();
            display_gas_detected();
            while (last_measure >= 70) {
                PORTB = 0xFF;
                for (unsigned char i = 255; i >= last_measure; i--) {
                    _delay_ms(1);
                }
                PORTB = 0x00;
                for (unsigned char i = 255; i >= last_measure; i--) {
                    _delay_ms(1);
                }
            }
            lcd_clear();
            display_clear();
        }
    }
}
```